ARCHITECTURES

Presented By:
DR. Ahmed Al-Badawy
Lecturer, Computer Science Department

LECTURE TOPICS

1. Distributed Systems & the Need for Architecture
2. Logical vs Physical Organization
3. Software Architecture Concepts
4. Components and Connectors
5. Architectural Styles Overview
6. Layered Architecture
7. Object-Based Architecture & Remote Invocation
8. Data-Centered Architecture
9. Event-Based Architecture
10. System Architectures
11. Client–Server & Multitier Architectures
12. Peer-to-Peer Systems & Overlay Networks
13. Distributed Hash Tables & Chord
14. Middleware & Distribution Transparency
15. Autonomic Systems & Self-Management
16. Chapter Summary

DISTRIBUTED SYSTEMS AND THE NEED FOR ARCHITECTURE

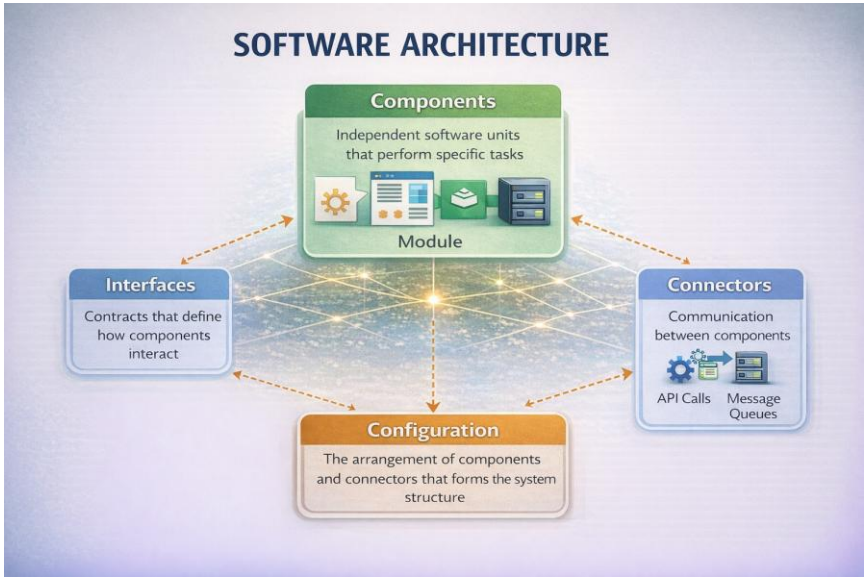
- Distributed systems consist of multiple software components running on different machines connected through a network.
- These components cooperate to perform tasks that appear to users as a single coherent system.
- Because components are physically separated, communication must occur through message passing or remote invocation mechanisms.
- Without a well-defined architecture, coordinating these distributed components becomes extremely complex.
- Architecture determines how responsibilities are divided among components in the system.
- It also defines how components communicate and how data flows between them.
- A good architecture improves scalability and allows systems to grow without major redesign.
- Architectural design decisions strongly influence performance, reliability, and maintainability.

LOGICAL ORGANIZATION VS PHYSICAL ORGANIZATION

- Distributed systems can be described from two complementary viewpoints: logical structure and physical deployment.
- Logical organization describes how software components are structured and how they interact conceptually.
- Physical organization describes how these components are mapped onto machines across the network.
- A system may appear logically simple while being physically distributed across many nodes.
- Designers must ensure that the logical design can be efficiently implemented in the physical infrastructure.
- Communication delays, network topology, and node distribution affect physical architecture decisions.
- Understanding both perspectives helps designers create scalable distributed solutions.
- Separating logical and physical views simplifies system analysis and design.

SOFTWARE ARCHITECTURE

- Software architecture describes the overall structure of a distributed system.
- It identifies system components and the responsibilities of each component.
- It also specifies connectors that allow components to communicate and cooperate.
- Connectors may represent message channels, procedure calls, event streams, or data pipelines.
- Architectural design helps organize complex systems into manageable modules.
- Components interact through well-defined interfaces rather than direct access to internal structures.
- This abstraction enables system evolution without requiring major changes to other components.
- As distributed systems grow larger, architectural design becomes increasingly important.



COMPONENTS IN DISTRIBUTED SYSTEMS

- A component is a modular software unit responsible for performing a specific function.
- Components encapsulate internal state and expose functionality through interfaces.
- Interfaces define how other components interact with the component.
- Because components communicate through interfaces, they can often be replaced independently.
- Distributed components may run on different machines within the network.
- Each component performs a well-defined role in the system architecture.
- Modular component design improves maintainability and system clarity.
- Many distributed frameworks are built around component-based architectures.

CONNECTORS BETWEEN COMPONENTS

- Connectors enable communication and coordination between system components.
- They represent the mechanisms through which components exchange information.
- Connectors may support synchronous or asynchronous communication models.
- Examples include remote procedure calls, message queues, and event notifications.
- Connectors may also provide additional services such as buffering and reliability.
- Middleware technologies often implement connectors used by distributed applications.
- Proper connector design ensures efficient communication across network boundaries.
- Choosing appropriate communication mechanisms is critical in distributed systems.

ARCHITECTURAL STYLES

- Architectural styles define common patterns used to organize distributed systems.
- Each style specifies types of components and allowed interactions between them.
- Styles provide design guidelines that simplify system construction.
- They help engineers reuse proven architectural approaches rather than designing systems from scratch.
- Architectural styles also influence system scalability and fault tolerance.
- Some styles are better suited for large-scale data sharing systems.
- Others are designed for modular service-based systems.
- Selecting the right architectural style is a key design decision.

MAJOR ARCHITECTURAL STYLES

- Distributed systems commonly use several fundamental architectural styles.
- Layered architectures organize system functionality into hierarchical layers.
- Object-based architectures focus on interacting objects and service invocation.
- Data-centered architectures rely on shared repositories for communication.
- Event-based architectures use event notifications to coordinate components.
- Each architectural style emphasizes different communication patterns.
- Designers often combine multiple styles in a single system.
- Hybrid designs are common in modern distributed platforms.

LAYERED ARCHITECTURE CONCEPT

- In a layered architecture, system components are organized into layers placed one above another.
- Each layer provides services to the layer above it.
- At the same time, each layer relies on services provided by the layer below.
- This hierarchical structure separates system functionality into different levels of abstraction.
- Higher layers focus on user-level functionality and application logic.
- Lower layers focus on low-level operations such as communication and resource management.
- This separation simplifies system design and maintenance.
- Layered architectures are widely used in operating systems and network protocols.

LAYER RESPONSIBILITIES

- Each layer in a layered architecture has clearly defined responsibilities.
- Lower layers handle fundamental operations such as data transmission and hardware interaction.
- Middle layers handle system services such as communication protocols.
- Higher layers provide application-level functionality to users or applications.
- The separation of responsibilities allows developers to focus on specific system aspects.
- Changes in one layer do not necessarily require modifications in other layers.
- This modular structure simplifies testing and debugging.
- Layer abstraction reduces system complexity.

ADVANTAGES OF LAYERED ARCHITECTURE

- Layered architectures promote modular system design.
- Developers can implement or modify layers independently.
- Errors can often be traced to a specific layer, simplifying debugging.
- System upgrades may only require modifying one layer.
- The architecture encourages separation of concerns in system design.
- Different development teams can work on different layers simultaneously.
- Reusable layers can be applied to multiple systems.
- These advantages explain why layered architectures are widely used.

OBJECT-BASED ARCHITECTURE

- Object-based architectures organize distributed systems as collections of interacting objects.
- Each object encapsulates both data and operations.
- Objects communicate by invoking operations on each other.
- These operations may involve remote communication when objects are located on different machines.
- Remote invocation mechanisms allow objects to interact across network boundaries.
- This approach promotes modular and reusable system design.
- Object interfaces define how services are accessed.
- Object-based systems are common in distributed middleware technologies.

REMOTE INVOCATION IN DISTRIBUTED OBJECTS

- When an object invokes a method on another object located remotely, network communication is required.
- The invocation must be translated into a network message sent to the remote machine.
- The remote system executes the requested operation.
- The result of the operation is then returned to the calling object.
- Remote invocation mechanisms hide network communication complexity from developers.
- Technologies such as RPC and RMI implement this communication model.
- These mechanisms allow distributed objects to interact as if they were local.
- This transparency simplifies application development.

DATA-CENTERED ARCHITECTURE

- Data-centered architectures focus on a shared data repository as the central system component.
- Processes communicate indirectly through this repository.
- Instead of interacting directly, components read and write shared data.
- The repository may store files, database records, or other shared resources.
- This architecture simplifies coordination between distributed components.
- The repository often acts as a central point for system consistency.
- Many distributed storage systems follow this architecture.
- Data management becomes the core responsibility of the system.

EVENT-BASED ARCHITECTURE

- Event-based architectures use events to coordinate distributed components.
- An event represents the occurrence of an action or change in the system.
- Components generate events to notify other components about system activities.
- Other components react to events based on their interests.
- Communication is typically asynchronous.
- Components do not need direct knowledge of other components.
- Event systems promote loose coupling between system modules.
- This architecture supports highly dynamic distributed systems.

SYSTEM ARCHITECTURES

- While architectural styles describe logical organization, system architectures describe physical deployment.
- System architecture defines how components are distributed across machines.
- It determines which nodes perform specific tasks.
- Communication patterns depend on the physical organization of nodes.
- System architecture influences system scalability and reliability.
- Designers must consider network latency and resource distribution.
- Different architectures provide different performance trade-offs.
- Understanding system architectures is essential for distributed system design.

CLIENT–SERVER ARCHITECTURE

- Client-server architecture divides the system into two main roles: clients and servers.
- Clients request services from servers.
- Servers provide services and manage system resources.
- Clients and servers communicate through network protocols.
- Servers may handle requests from many clients simultaneously.
- This architecture simplifies system organization.
- Many Internet services use the client-server model.
- Examples include web servers and database servers.

MULTITIER ARCHITECTURES

- Multitier architectures extend the client-server model by introducing additional layers.
- Each tier performs a specialized role in system processing.
- The presentation tier handles user interfaces.
- The application tier implements business logic.
- The data tier manages persistent storage.
- This separation improves scalability and maintainability.
- Systems can scale by adding servers to specific tiers.
- Many modern web applications follow this architecture.

PEER-TO-PEER ARCHITECTURE

- Peer-to-peer architectures remove the strict distinction between clients and servers.
- Each node can act both as a client and as a server.
- Nodes cooperate by sharing resources such as storage and processing power.
- This architecture distributes system workload across many nodes.
- Peer-to-peer systems are highly scalable.
- They can operate without centralized infrastructure.
- However, managing coordination between nodes becomes more complex.
- Many large-scale distributed applications use peer-to-peer techniques.

OVERLAY NETWORKS

- Peer-to-peer systems organize nodes using overlay networks.
- An overlay network is a logical network built on top of the physical network.
- Nodes maintain connections to selected peers rather than all nodes.
- These connections define routing paths used to locate resources.
- Overlay networks allow systems to scale to thousands or millions of nodes.
- They provide efficient resource discovery mechanisms.
- Overlay structures often follow specific algorithms.
- Structured overlays support deterministic routing.

DISTRIBUTED HASH TABLES

- Structured peer-to-peer systems frequently use Distributed Hash Tables (DHTs).
- A DHT assigns identifiers to nodes and keys to data items.
- Data items are stored on nodes responsible for specific keys.
- When a node wants to retrieve data, it calculates the key of that data item.
- The system routes the request toward the node responsible for that key.
- Routing algorithms ensure efficient lookup across the network.
- Most DHT systems locate data in logarithmic time relative to the number of nodes.
- This approach allows scalable distributed storage systems.

CHORD SYSTEM

- Chord is a well-known Distributed Hash Table implementation.
- Nodes are organized in a logical ring structure.
- Each node is responsible for a range of keys in the identifier space.
- Data items are stored at nodes responsible for their keys.
- Lookup operations locate the node responsible for a specific key.
- Nodes maintain routing tables to accelerate search operations.
- These routing tables allow efficient navigation of the ring structure.
- Chord demonstrates how DHT systems achieve scalable resource lookup.

MIDDLEWARE

- Middleware provides a software layer between distributed applications and underlying operating systems.
- It simplifies communication between distributed components.
- Middleware hides many complexities associated with distributed computing.
- Developers interact with high-level abstractions rather than low-level network protocols.
- Middleware often provides communication frameworks and service discovery.
- It may also support transaction management and security services.
- Many distributed platforms rely on middleware technologies.
- Middleware enables interoperability between heterogeneous systems.

DISTRIBUTION TRANSPARENCY

- Distribution transparency hides the distributed nature of a system from users and applications.
- Applications interact with the system as if it were a single unified environment.
- Transparency mechanisms mask differences between local and remote resources.
- Middleware often implements these transparency features.
- Transparency simplifies application development.
- However, excessive transparency may hide performance limitations.
- Designers must balance transparency with system efficiency.
- Understanding these trade-offs is important in distributed system design.

AUTONOMIC SYSTEMS

- Autonomic systems are capable of managing themselves with minimal human intervention.
- They continuously monitor system behavior and environmental conditions.
- Based on monitoring data, they adjust system operation automatically.
- This concept is inspired by the human autonomic nervous system.
- Autonomic systems reduce administrative overhead in large distributed environments.
- They improve system reliability and resilience.
- Self-management is increasingly important in cloud computing infrastructures.
- Autonomic architectures support dynamic adaptation.

SELF-MANAGEMENT CAPABILITIES

- Self-configuration enables systems to configure themselves automatically.
- Self-optimization improves system performance during operation.
- Self-healing detects and repairs system failures.
- Self-protection defends the system against malicious attacks.
- These capabilities reduce the need for manual system management.
- They allow distributed systems to operate efficiently at large scale.
- Monitoring and analysis components support these capabilities.
- Feedback loops control system adaptation.

CHAPTER SUMMARY

- Distributed system architecture determines how components interact and cooperate.
- Architectural styles provide patterns for organizing distributed components.
- System architectures define how components are deployed across network nodes.
- Client-server and peer-to-peer architectures represent two major system models.
- Overlay networks and distributed hash tables enable scalable resource discovery.
- Middleware simplifies communication and system integration.
- Distribution transparency hides complexity from users and developers.
- Autonomic systems enable self-management in large distributed infrastructures.